

目 录

第 1 章 实验概述	1
1.1 实验目的	1
1.2 实验环境	1
第 2 章 实验内容和步骤	2
2.1 启动 MIPSsim	2
2.2 流水线各段功能理解	2
2.2.1 IF/ID 段	2
2.2.2 ID/EX 段	3
2.2.3 EX/MEM 段	3
2.2.4 MEM/WB 段	4
2.3 载入一个样例程序	5
2.4 观察和分析结构冲突对 CPU 性能的影响	9
2.5 观察数据冲突并用定向技术	13
第 3 章 实验总结与心得体会	16

第 1 章 实验概述

1.1 实验目的

1. 加深对计算机流水线基本概念的理解。
2. 理解 MIPS 结构如何用 5 段流水线来实现，理解各段的功能和基本操作。
3. 加深对数据冲突和资源冲突的理解，理解这两类冲突对 CPU 性能的影响。
4. 进一步理解解决数据冲突的方法，掌握如何应用定向技术来减少数据冲突引起的停顿。

1.2 实验环境

表 1.1 实验环境

配置项	参数
操作系统	Windows 11 24H2
仿真工具	MIPSSim v1.0.0 64-bit

第 2 章 实验内容和步骤

2.1 启动 MIPSsim

双击 MIPS 模拟器 (64 位).exe, 打开 MIPSsim 模拟器, 进入主界面。

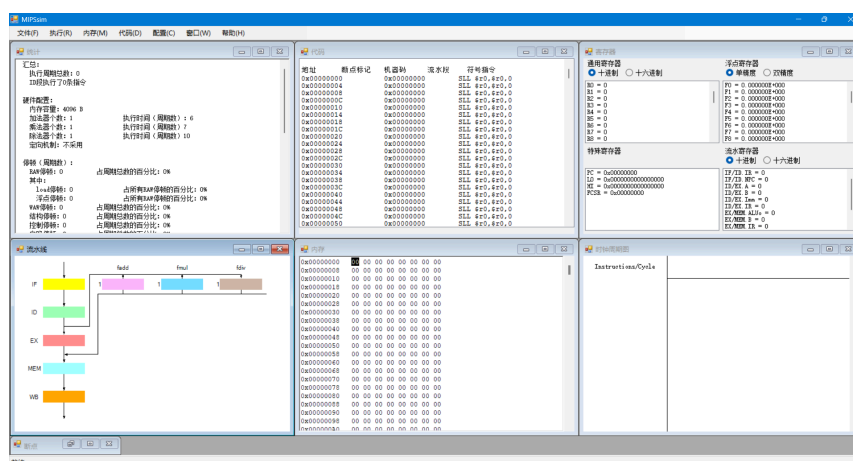


图 2.1 MIPSsim 主界面

2.2 流水线各段功能理解

通过鼠标双击各段, 了解各流水寄存器的内容及对应作用。

2.2.1 IF/ID 段

IF/ID 段的主要功能是从存储器取指令, 并将取出的指令及下一条指令地址传递给 ID 段。如图 2.2 所示, 本段对应的流水寄存器包括:

1. IR: 指令寄存器, 存放从存储器取出的当前指令的机器码。
2. NPC: 下一程序计数器, 存放下一条指令的地址。
3. RET: 返回地址, 存放过程调用返回地址等信息, 在流水线中逐段传递。

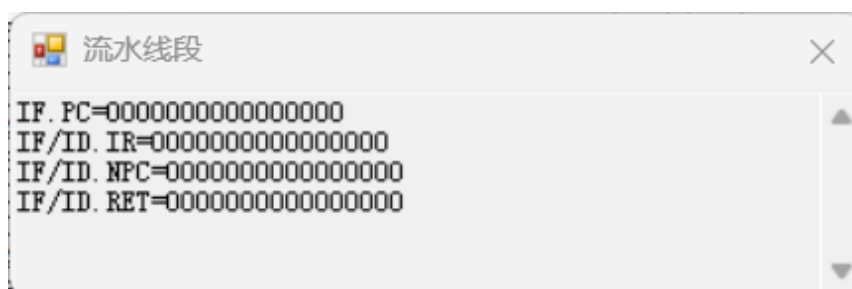


图 2.2 IF/ID 段流水线内容

2.2.2 ID/EX 段

ID/EX 段的主要功能是对指令进行译码，读取寄存器值并进行立即数扩展，将操作数传递给 EX 段。如图 2.3 所示，本段对应的流水寄存器包括：

1. A：送往 EX 段的第一个操作数，通常由 OP1 整理后得到，是 ALU 的一个主要输入。
2. B：送往 EX 段的第二个操作数，供算术逻辑运算或存储写数使用。
3. IMM：扩展后的立即数，供 EX 段在立即数运算、地址计算中直接使用。
4. IR：同上一阶段。
5. RET：同上一阶段。

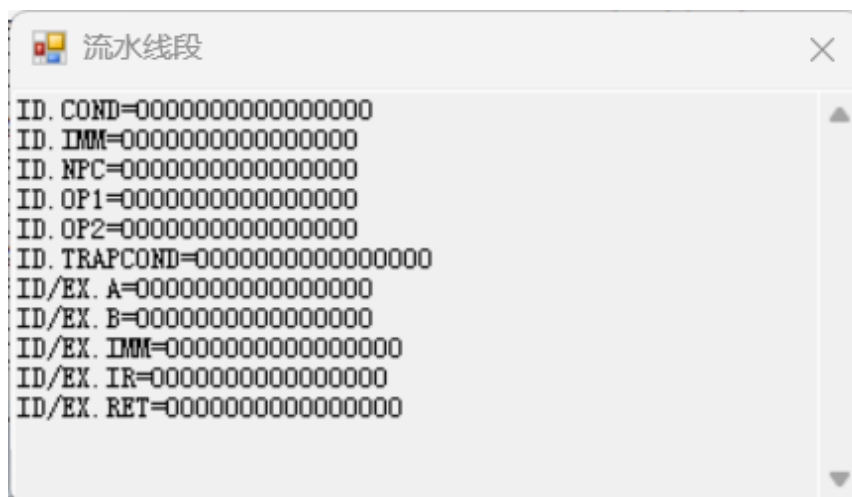


图 2.3 ID/EX 段流水线内容

2.2.3 EX/MEM 段

EX/MEM 段的主要功能是执行 ALU 运算或地址计算，将运算结果和待存储数据传递给 MEM 段。如图 2.4 所示，本段对应的流水寄存器包括：

1. ALUOUT：存储 ALU 运算结果，既可以是算术逻辑结果，也可以是 load/store 指令的有效地址。
2. B：从前级保留下来的第二操作数。
3. FOUT：用于保存浮点运算结果或标志位。
4. IR：同上一阶段。
5. RET：同上一阶段。

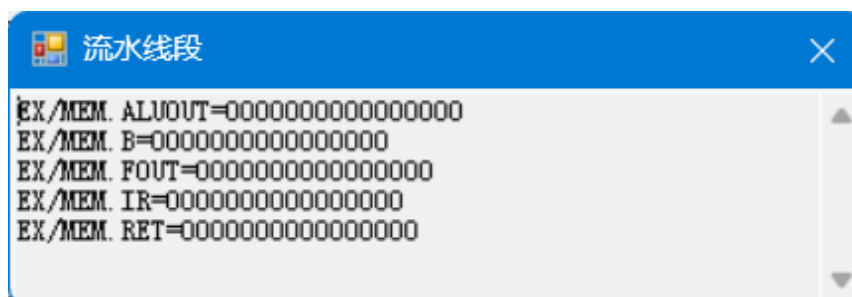


图 2.4 EX/MEM 段流水线内容

2.2.4 MEM/WB 段

MEM/WB 段的主要功能是完成存储器访问（读或写），将结果传递给 WB 段以写回寄存器。如图 2.5 所示，本段对应的流水寄存器包括：

1. ALUOUT：同上一阶段。
2. FOUT：同上一阶段。
3. IR：同上一阶段。
4. LMD：存放从存储器读出的数据。
5. RET：同上一阶段。

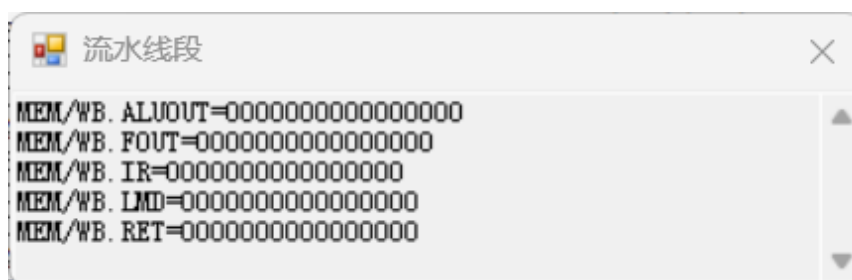


图 2.5 MEM/WB 段流水线内容

2.3 载入一个样例程序

1. 点击菜单栏的 文件 → 载入程序，在弹出的文件选择框中找到并选择 branch.s 文件，点击打开。
2. 程序载入完成后，确认代码窗口已经显示 branch.s 中的指令序列，PC 指向程序入口位置，寄存器窗口和存储器窗口已处于可观察状态。
3. **单步执行一个周期：** 点击菜单栏的 执行 → 单步执行一个周期，使处理器向前执行 1 个时钟周期，如图 2.6 所示。

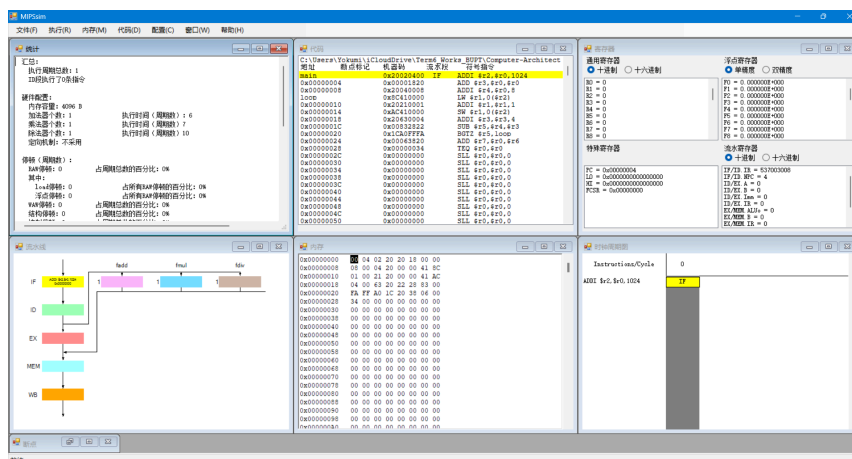


图 2.6 单步执行第一个周期后的界面状态

4. 单步执行到第三个周期，此时程序正在进行第三条指令 ADDI \$r4,\$r0,8 的 IF 阶段，第一条指令 ADDI \$r2,\$r0,1024 已进入 EX 阶段，如图 2.7 所示的时钟周期图所示。通过观察及存取窗口和代码窗口也可以验证，此时 Imm = 1024，对应第一条指令的立即数，而 IR 中的指令机器码也对应着第三条指令。

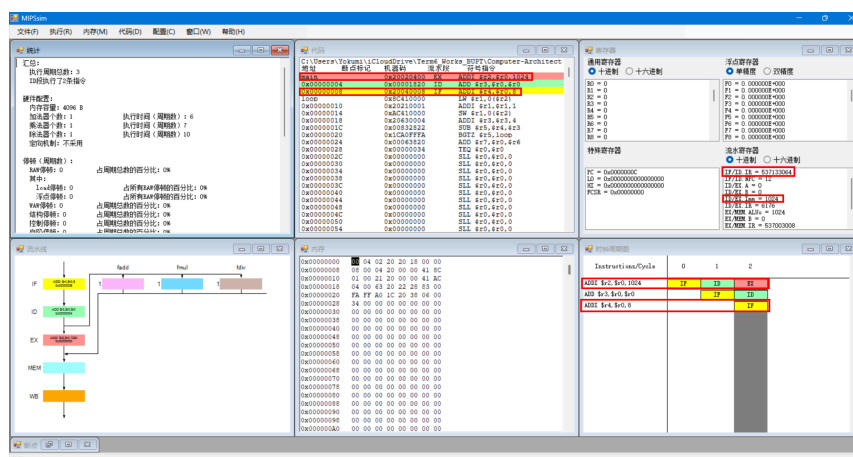


图 2.7 单步执行到第三个周期的界面状态

5. 单步执行到第七个周期，此时由于第四条指令 `LW $r1, 0($r2)` 未完成 WB，导致后面 2 条指令 `ADDI $r1, $r1, 1` 和 `SW $r1, 0($r2)` 均产生停顿，等待 LW 指令完成写回后才能继续执行，如图 2.8 所示。

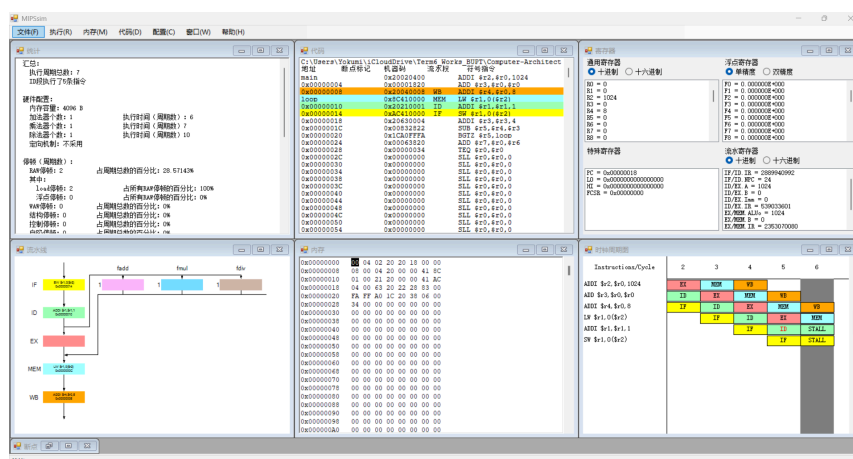


图 2.8 单步执行到第七个周期的界面状态

4. **执行多个周期：**点击菜单栏的执行 → 执行多个周期，在弹出的对话框中输入需要连续执行的周期数，这里选择执行到第 20 个周期，如图 2.9 所示，此时尚未满足退出循环的条件，程序重新跳转到 loop 标签处继续执行。

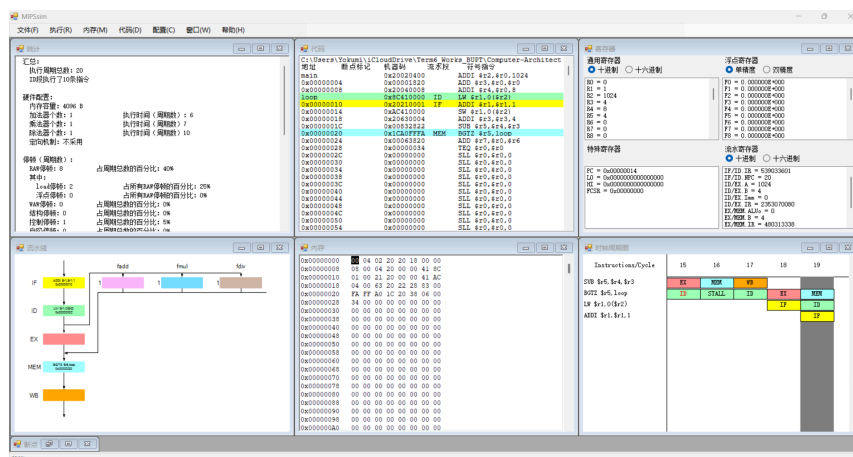


图 2.9 执行多个周期到第 20 个周期的界面状态

5. 执行多个周期到第 34 个周期，如图 2.10 所示，此时 $r5 = 0 \leq 0$ ，满足退出循环的条件，程序跳出循环继续执行后续指令。

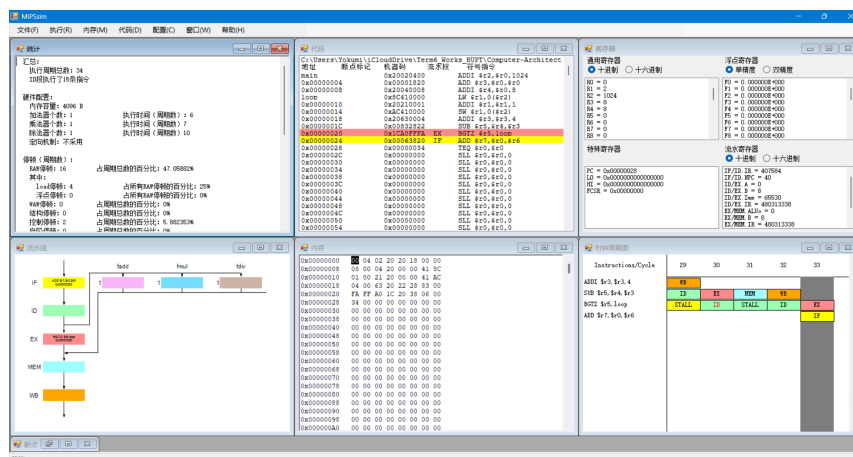


图 2.10 执行多个周期到第 34 个周期的界面状态

6. 执行多个周期到程序结束，即第 38 个周期，如图 2.11 所示，此时程序因 `TEQ $r0, $r0` 指令触发了异常而停止，界面给出相关提示信息。

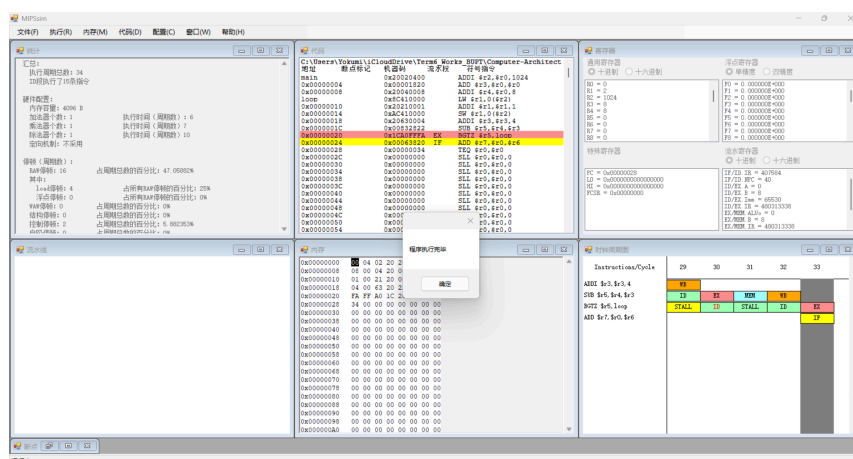


图 2.11 单步执行到程序结束的界面状态

7. **连续执行**：点击菜单栏的执行 → 连续执行，使程序持续运行，直到程序结束、命中异常或被用户主动停止，如图 2.12 所示，共发生了 16 次 RAW 停顿，其中 4 次是 load 停顿。通过我们前面单步执行的观察也可以验证，大部分停顿均由**数据相关**产生。

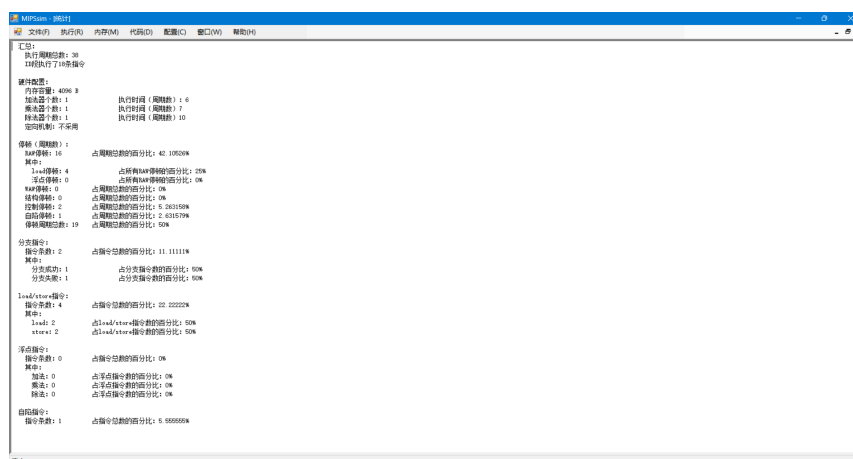


图 2.12 连续执行后的界面状态

8. **设置断点观察程序执行**：在代码窗口中选择关键指令所在行设置断点，这里选择 `ADDI $r3, $r3, 4` 这条指令，如图 2.13 所示，设置断点后该行会高亮显示。

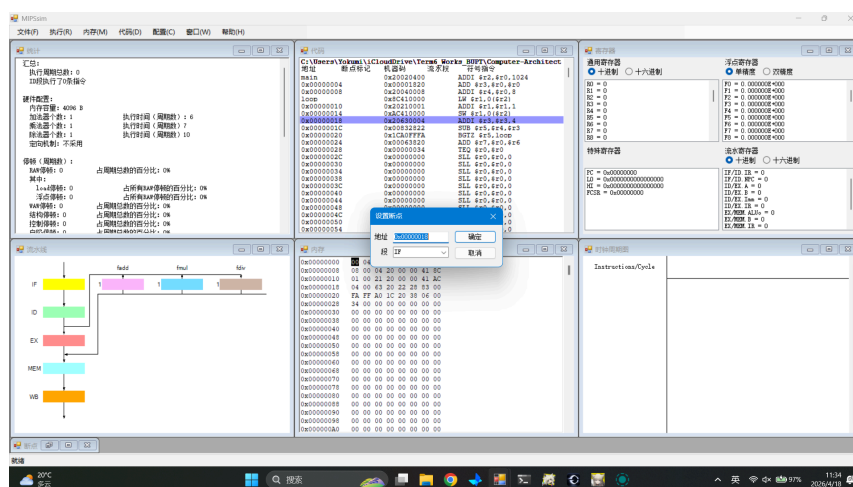


图 2.13 设置断点

9. 设置断点后, 点击执行 → 连续执行, 使程序自动运行到断点位置并暂停, 如图 2.14 和图 2.15 所示, 程序分别在第 9 个周期和第 24 个周期停在断点处。

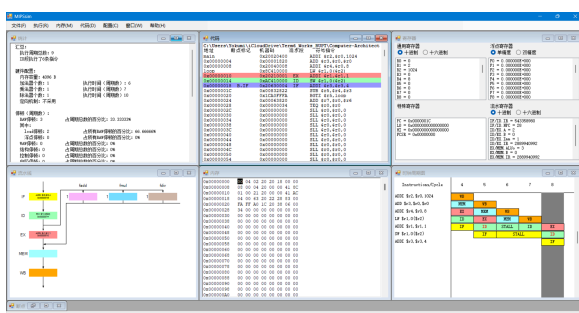


图 2.14 程序在第 9 个周期停在断点处

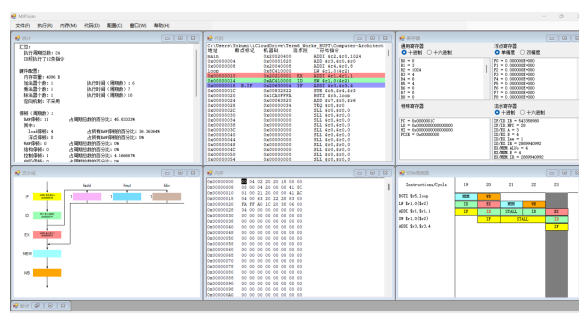


图 2.15 程序在第 24 个周期停在断点处

10. 在菜单栏点击配置 → 流水方式, 观察程序在流水方式下的执行情况。

2.4 观察和分析结构冲突对 CPU 性能的影响

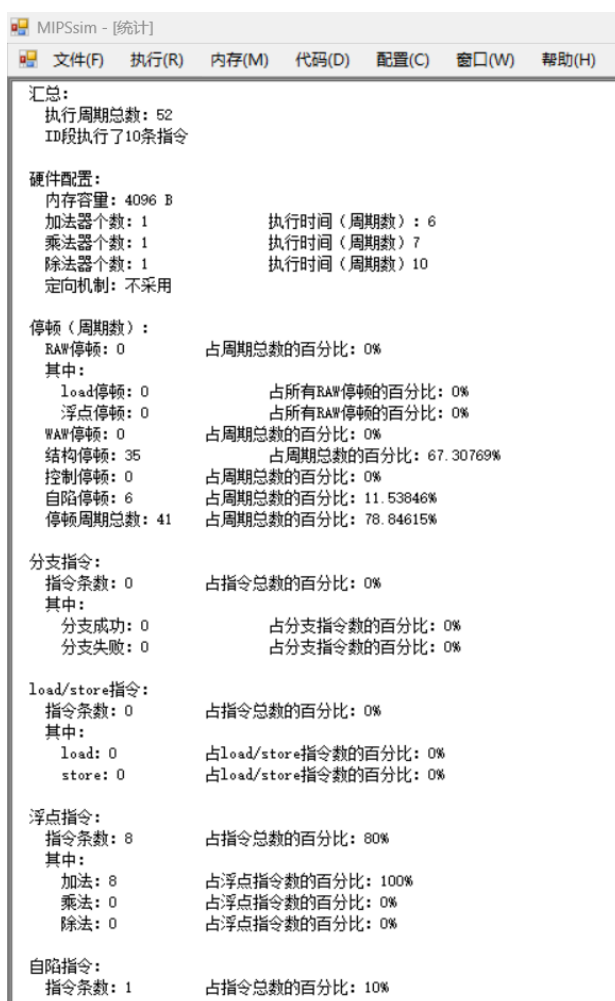
1. 点击菜单栏的文件 → 载入程序, 在弹出的文件选择框中找到并选择 structure_hz.s 文件, 点击打开。
2. 观察代码区域中的指令序列, 如图 2.16 所示, 程序中任何两条加法指令间都存在冲突, 导致冲突部件为加法部件。



地址	断点标记	机器码	流水段	符号指令
main		0x46210080		ADD.D \$f2, \$f0, \$f1
0x00000004		0x462100C0		ADD.D \$f3, \$f0, \$f1
0x00000008		0x46210100		ADD.D \$f4, \$f0, \$f1
0x0000000C		0x46210140		ADD.D \$f5, \$f0, \$f1
0x00000010		0x46210180		ADD.D \$f6, \$f0, \$f1
0x00000014		0x462101C0		ADD.D \$f7, \$f0, \$f1
0x00000018		0x46210200		ADD.D \$f8, \$f0, \$f1
0x0000001C		0x46210240		ADD.D \$f9, \$f0, \$f1

图 2.16 冲突代码段

3. 点击菜单栏的 执行 → 连续执行，观察程序执行情况，如图 2.17 所示。



汇总:	
执行周期总数:	52
ID段执行了10条指令	
硬件配置:	
内存容量:	4096 B
加法器个数:	1
乘法器个数:	1
除法器个数:	1
定向机制:	不采用
执行时间 (周期数):	6
执行时间 (周期数):	7
执行时间 (周期数):	10
停顿 (周期数):	
RAW停顿:	0
占周期总数的百分比:	0%
其中:	
Load停顿:	0
占所有RAW停顿的百分比:	0%
浮点停顿:	0
占所有RAW停顿的百分比:	0%
WAW停顿:	0
占周期总数的百分比:	0%
结构停顿:	35
占周期总数的百分比:	67.30769%
控制停顿:	0
占周期总数的百分比:	0%
自陷停顿:	6
占周期总数的百分比:	11.53846%
停顿周期总数:	41
占周期总数的百分比:	78.84615%
分支指令:	
指令条数:	0
占指令总数的百分比:	0%
其中:	
分支成功:	0
占分支指令数的百分比:	0%
分支失败:	0
占分支指令数的百分比:	0%
load/store指令:	
指令条数:	0
占指令总数的百分比:	0%
其中:	
load:	0
占load/store指令数的百分比:	0%
store:	0
占load/store指令数的百分比:	0%
浮点指令:	
指令条数:	8
占指令总数的百分比:	80%
其中:	
加法:	8
占浮点指令数的百分比:	100%
乘法:	0
占浮点指令数的百分比:	0%
除法:	0
占浮点指令数的百分比:	0%
自陷指令:	
指令条数:	1
占指令总数的百分比:	10%

图 2.17 程序执行情况

4. 由图 2.17 可知，程序总执行周期数为 52 个周期，其中由结构冲突引起的停顿周期数为 35，其占总执行周期数的百分比为 67.3%。

5. **增加浮点加法器**：点击菜单栏的 配置 → 常规配置，在弹出的对话框中把浮点加法器的个数改为 4 个，如图 2.18 所示。



图 2.18 增加浮点加法器

6. 重复上述步骤，观察程序执行情况，如图 2.19 所示。

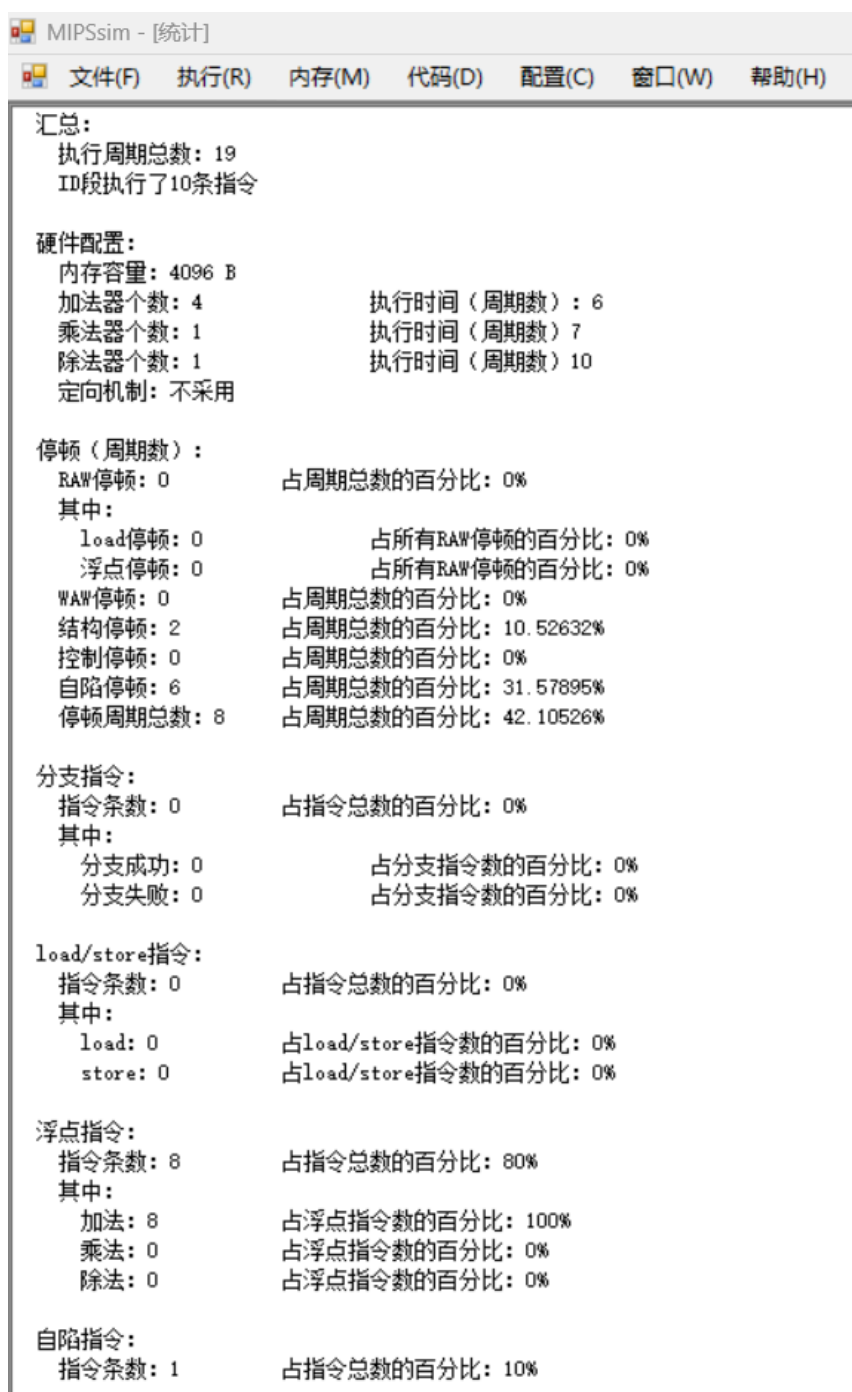


图 2.19 增加浮点加法器后的程序执行情况

7. 由图 2.19 可知, 程序总执行周期数为 19 个周期, 其中由结构冲突引起的停顿周期数为 2, 其占总执行周期数的百分比为 10.5%。与之前相比, 增加浮点加法器后结构冲突引起的停顿周期数和占比均大幅下降, 程序性能得到显著提升。

结论：结构冲突会严重影响 CPU 性能，增加相关功能部件的数量可以有效减少结构冲突引起的停顿，从而提升程序性能。

2.5 观察数据冲突并用定向技术

1. 全部复位。
2. 点击菜单栏的 文件 → 载入程序，在弹出的文件选择框中找到并选择 data_hz.s 文件，点击打开。
3. **关闭定向功能：**点击菜单栏的 配置，取消勾选 定向。
4. 用单步执行一个周期的方式执行该程序，通过观察时钟周期图，可以发现周期 4, 6, 7, 9, 10, 13, 14, 17, 18, 20, 21, 25, 26, 28, 29, 32, 33, 36, 37, 39, 40, 44, 45, 47, 48, 51, 52, 55, 56, 58, 59 共 31 次 RAW 停顿，如图 2.20 所示。

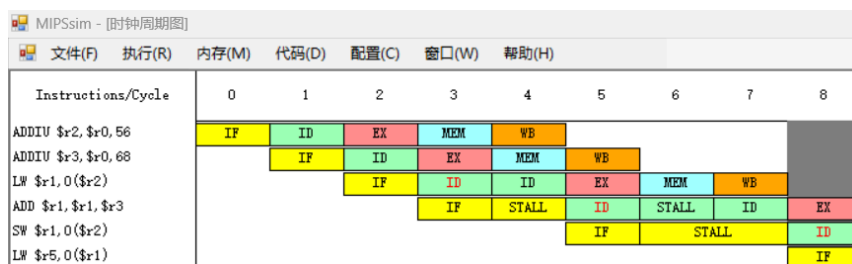


图 2.20 RAW 停顿情况

5. 观察统计结果，如图 2.21 所示，数据冲突引起的停顿周期数为 31 个周期，程序执行的总时钟周期数为 65 个周期，数据冲突引起的停顿占总执行周期数的百分比为 47.7%。

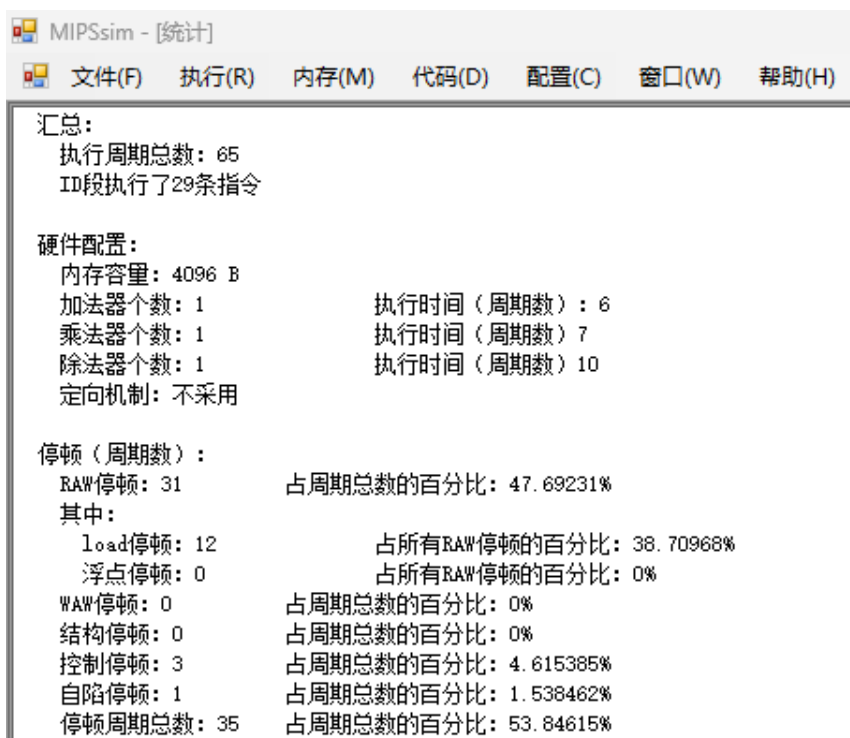


图 2.21 数据冲突统计结果

6. 复位 CPU。

7. 启用定向功能: 点击菜单栏的 配置, 勾选 定向。

8. 重复上述步骤, 观察程序执行情况, 此时发生周期 5、10、13、18、22、25、30、34、37 共 9 次 RAW 停顿, 如图 2.22 所示。

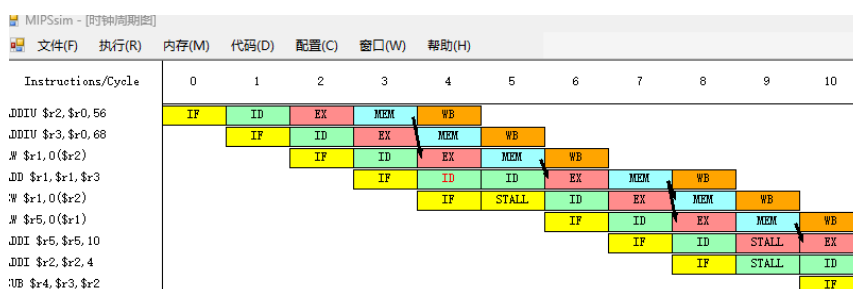


图 2.22 启用定向功能后的 RAW 停顿情况

9. 观察统计结果, 如图 2.23 所示, 数据冲突引起的停顿周期数为 9 个周期, 程序执行的总时钟周期数为 43 个周期, 数据冲突引起的停顿占总执行周期数的百分比为

20.9%。采用定向技术后，性能比原来提高了 $65 / 43 = 1.51$ 倍，数据冲突引起的停顿周期数和占比均大幅下降，程序性能得到显著提升。

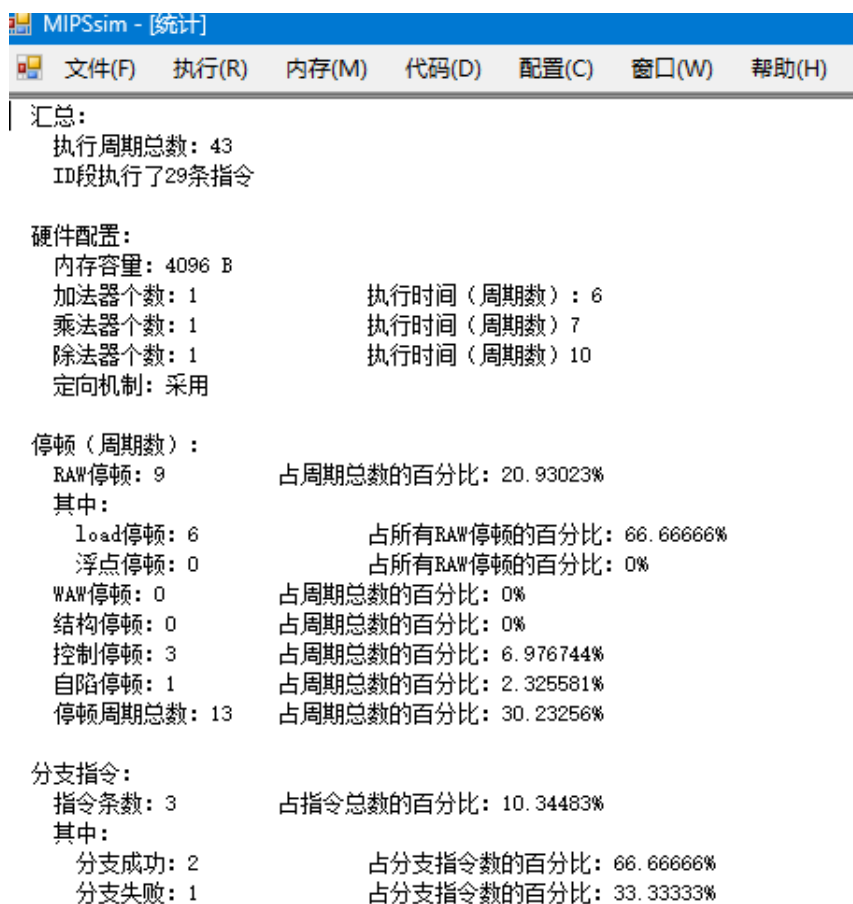


图 2.23 启用定向功能后的数据冲突统计结果

第3章 实验总结与心得体会

本实验通过 MIPSsim 模拟器深入理解了 MIPS 结构的 5 段流水线工作原理，通过对数据冲突和结构冲突进行了分析，我对流水线技术的优势与挑战有了更直观的认识：虽然流水线可实现指令级并行，但数据冲突和资源冲突会使其性能大打折扣。理解这些冲突的成因及解决方法至关重要。定向技术能有效减少大部分数据相关引起的停顿，使流水线接近理想状态，这是现代 CPU 普遍采用定向机制的原因。